Surfaces implicites

Saidoune Elyas et Lepers Leon

Master 2 Informatique – 2026–2027

Table des matières

1 Introduction **3**
1.1 Objectifs et réalisations . . . 3
1.2 Environnement . . . 3
1.3 Mesures de performances . . . 3

2 Modélisation par SDF **3**
2.1 Arbre de construction . . . 3
2.2 Primitives . . . 3
2.2.1 Sphère . . . 4
2.2.2 Box . . . 4
2.2.3 Plan . . . 5
2.2.4 Cylindre . . . 5
2.2.5 Capsule . . . 6
2.2.6 Cone . . . 6
2.2.7 Donut . . . 7
2.3 Résultats et performances . . . 7
2.4 Opérateurs et transformations . . . 7
2.4.1 Union . . . 8
2.4.2 Intersection . . . 8
2.4.3 Soustraction . . . 8
2.4.4 Blend . . . 9

3 Érosion procédurale **9**
3.1 Intersection rayon–SDF . . . 9
3.2 Algorithme d'érosion . . . 9
3.2.1 Projection des impacts . . . 9
3.2.2 Érosion incrémentale et par paquets . . . 10
3.3 Résultats . . . 10
3.3.1 Analyse : incrémental contre paquets . . . 10

4 Accélération des requêtes **12**
4.1 Structure et stratégie implémentées 12
4.2 Résultats 12

5 Rendu par lancer de rayon **12**
5.1 Sphere Tracing 12
5.2 Applatissement de l'abre 13

A Code source **13**

B Aide et ressources **14**

1 Introduction

1.1 Objectifs et réalisations

Pour ce devoir nous avons abordé trois des quatre sujets : la modélisation de Signed Distance Field, l'algorithme d'érosion ainsi que le volume englobant. En plus de ces trois sujets nous avons mis en place une méthode de visualisation supplémentaire à celle fournie utilisant le marching cube. Nous avons développé une visualisation en temps réel via lancer de rayon et sphère tracing. Ce supplément est décrit dans sa partie ainsi que l'aplatissement des arbres en GLSL.

1.2 Environnement

Le projet a été réalisé et testé sous Linux en C++, avec OpenGL pour la visualisation. Le projet reprend le code de marching cube fourni. Pour plus de précision sur l'installation et la compilation référez vous au README.MD du projet.

1.3 Mesures de performances

Pour mesurer les performances nous avons pris la mesure de la durée des appels aux fonctions Value et Intersect et nous nous sommes également basés sur les images par seconde ainsi que le temps de calcul d'une image. Cette dernière valeur est pertinente car la scène est calculée en temps réel avec des appels aux fonctions Value et avec des lancers de rayon mais elle est à nuancer car sa valeur finale est influencée par le nombre de pixels occupés par l'objet. Sa mesure a donc été prise avec précaution en essayant de garder une surface occupée à l'écran semblable pour chaque primitive.

2 Modélisation par SDF

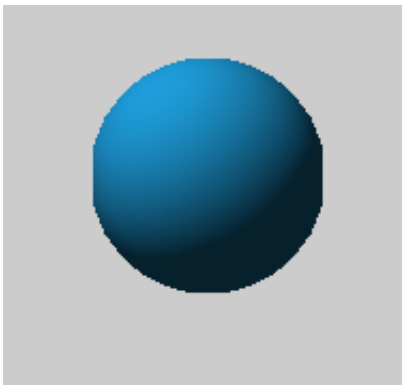
2.1 Arbre de construction

Les scènes sont représentées sous forme d'arbres. Chaque primitive ainsi que chaque opérateur hérite de la classe NodeSDF, les opérateurs sont des nœuds parents tandis que les primitives sont des feuilles. L'ensemble des primitives et opérateurs est défini dans le fichier ***SDF.cpp***.

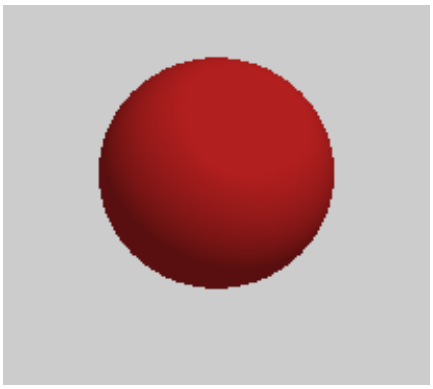
2.2 Primitives

Ci-dessous vous trouverez l'ensemble des primitives que nous avons définies. Pour chacune d'entre elles, une image générée par marching cube (elle y apparaît en bleu) et une autre image générée par sphere tracing (elle y apparaît en rouge) sont présentées avec un tableau qui détaille les performances. Ici les FPS et le temps par frame sont calculés pour 500 exemplaires d'une même primitive placés au même endroit (et donc occupant chacun le même nombre de pixels) et pour le temps d'évaluation de Value il est moyenné sur 10^7 évaluations. Dans le cas du plan, NR signifie "non représentatif", car il prend une trop grande partie de l'écran.

2.2.1 Sphère



Marching cubes



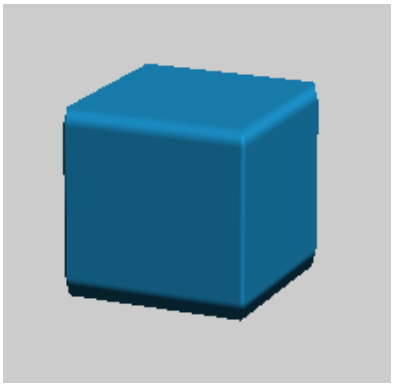
Sphere tracing

Figure 1 – Deux vues de la primitive Sphère.

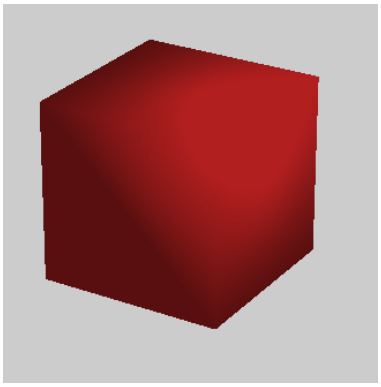
Table 1 – Performances d'évaluation de la primitive Sphère.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Sphère	40.0	24.9	13.6

2.2.2 Box



Marching cubes



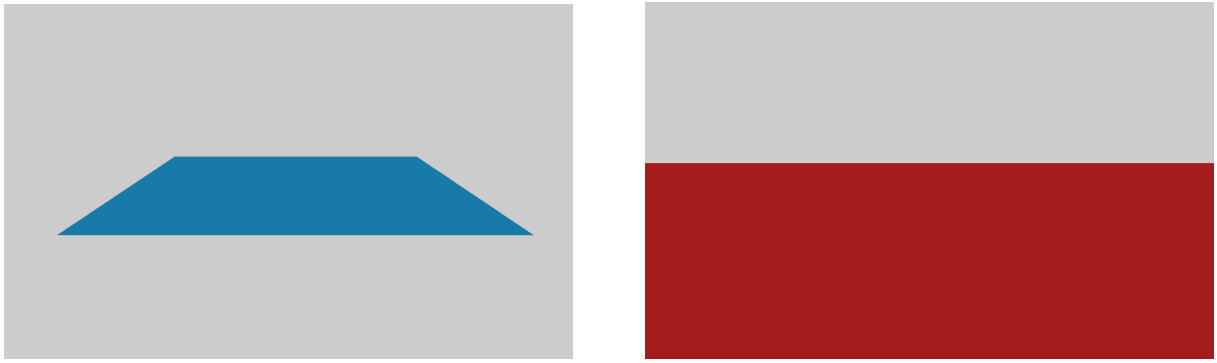
Sphere tracing

Figure 2 – Deux vues de la primitive Box.

Table 2 – Performances d'évaluation de la primitive Box.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Box	24.6	40.6	49.6

2.2.3 Plan



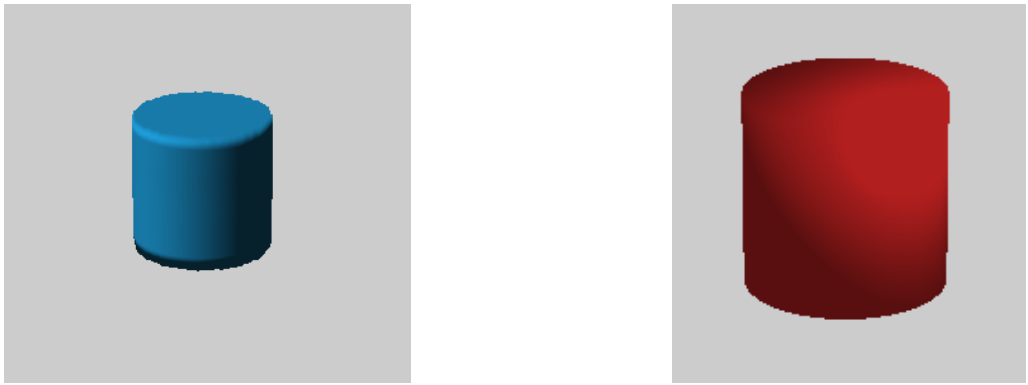
Marching cubes Sphere tracing

Figure 3 – Deux vues de la primitive Plan.

Table 3 – Performances d'évaluation de la primitive Plan.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Plan	NR	NR	14.6

2.2.4 Cylindre



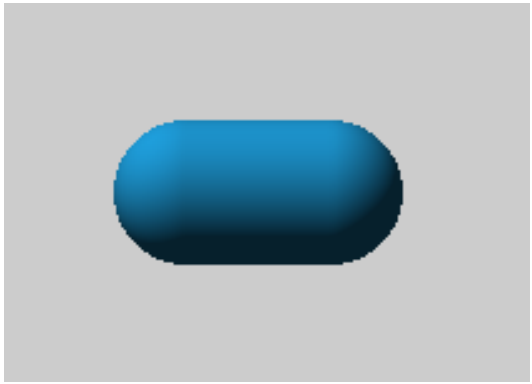
Marching cubes Sphere tracing

Figure 4 – Deux vues de la primitive Cylindre.

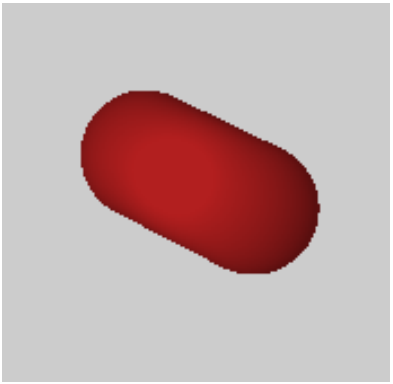
Table 4 – Performances d'évaluation de la primitive Cylindre.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Cylindre	32.9	30.38	31.6

2.2.5 Capsule



Marching cubes



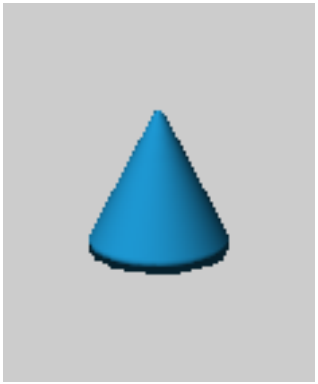
Sphere tracing

Figure 5 – Deux vues de la primitive Capsule.

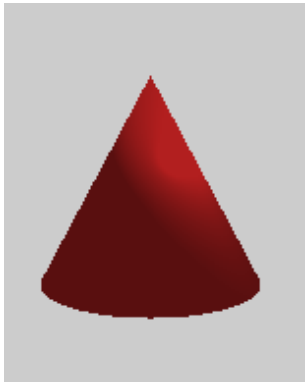
Table 5 – Performances d'évaluation de la primitive Capsule.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Capsule	39.8	25.24	18.7

2.2.6 Cone



Marching cubes



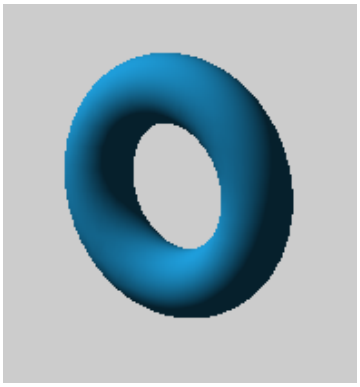
Sphere tracing

Figure 6 – Deux vues de la primitive Cone.

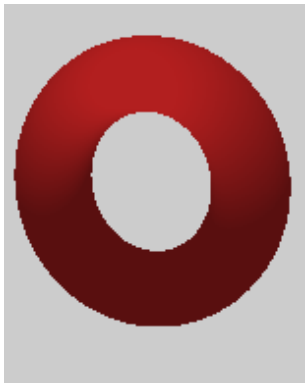
Table 6 – Performances d'évaluation de la primitive Cone.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Cone	16	62.58	48.05

2.2.7 Donut



Marching cubes



Sphere tracing

Figure 7 – Deux vues de la primitive Donut.

Table 7 – Performances d'évaluation de la primitive Donut.

Primitive	FPS	Temps/frame (ms)	Temps moyen de `Value` (μs/appel)
Donut	41.0	24.43	14.8

2.3 Résultats et performances

Temps relatif : évaluation de `Value` / évaluation de la sphère

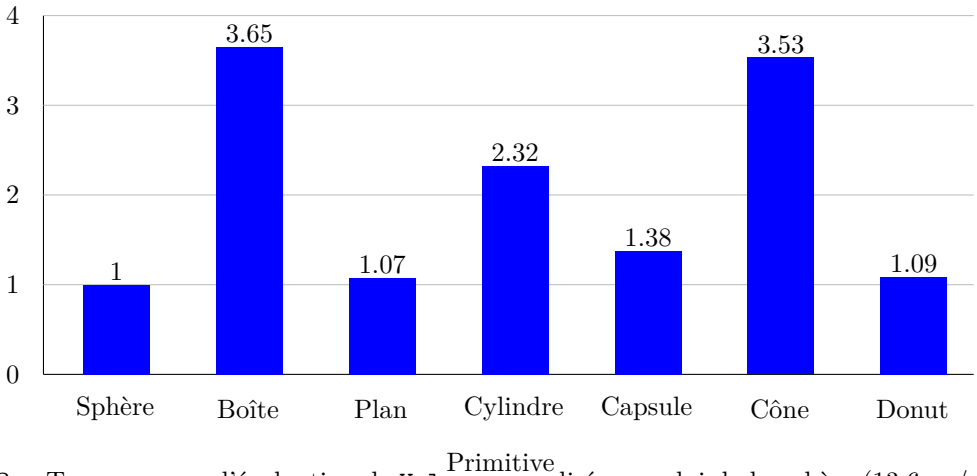


Figure 8 – Temps moyen d'évaluation de `Value`, normalisé par celui de la sphère (13.6 μs/appel).

La figure ci-dessus utilise la sphère comme primitive de référence, nous la considérons comme la plus "simple". On peut y voir que le plan ainsi que le donut sont sensiblement similaires à la sphère. La boite est la primitive la plus longue à calculer car nous l'avons définie de sorte à pouvoir lui appliquer facilement une rotation. Les calculs supplémentaires peuvent donc expliquer ses performances.

2.4 Opérateurs et transformations

Ci-dessous des images des quatre opérateurs définis dans le projet.

2.4.1 Union

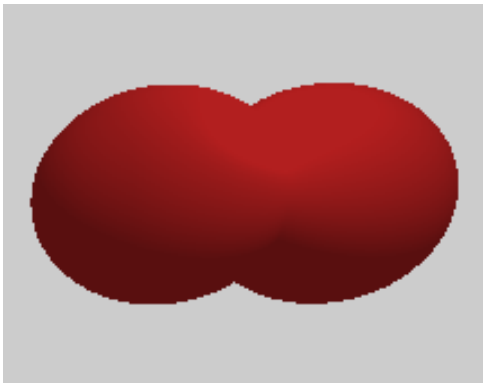


Figure 9 – Résultat de l'union.

2.4.2 Intersection

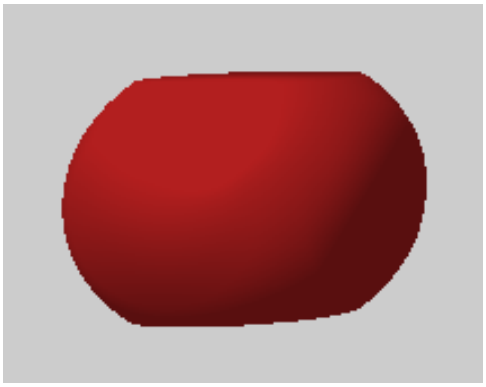


Figure 10 – Résultat de l'intersection entre un cube et une sphère.

2.4.3 Soustraction

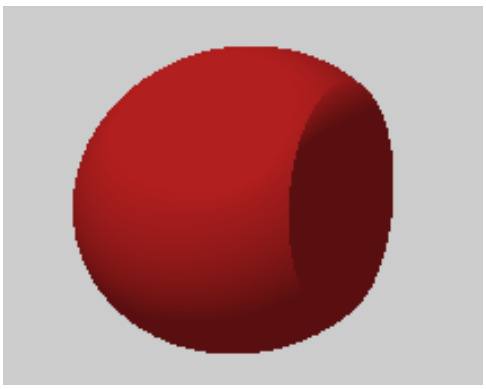


Figure 11 – Résultat de la soustraction.

2.4.4 Blend

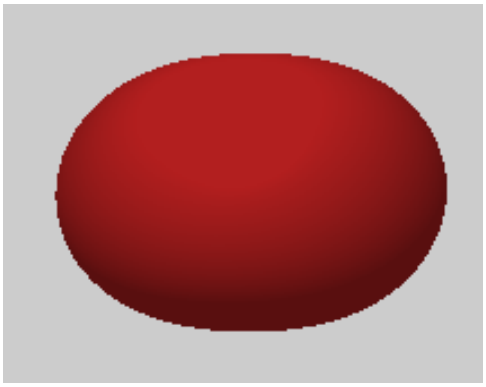


Figure 12 – Résultat du blend.

3 Érosion procédurale

L'objectif est de simuler le vieillissement d'un objet : on projette sur lui des sphères qui creusent localement sa surface. Toute l'érosion est faite côté CPU sur l'arbre C++ (fichiers `SI/erosion.hpp` et `SI/erosion.cpp`) ; les mesures sont dans un programme séparé (`erosion_benchmark.cpp`).

3.1 Intersection rayon–SDF

L'intersection est la méthode `bool NodeSDF::Intersect(const Ray& ray, double& t, double tMax, double eps, int maxSteps) const`, disponible pour tous les nœuds de l'arbre. Elle utilise le sphere tracing : en un point p, la valeur $d = f(p)$ du champ est la distance à la surface la plus proche, donc on peut avancer de d le long du rayon sans traverser la surface.
On part de l'origine du rayon ($t = 0$) puis, à chaque itération :

1. on calcule $p = o + t\,\vec{u}$ et $d = f(p)$;
2. si $d < \varepsilon$ ($\varepsilon = 10^{-4}$), on est sur la surface : on renvoie `true` et t contient la distance d'impact ;
3. sinon on avance : $t \leftarrow t + d$;
4. si $t > t_{\max}$ ($t_{\max} = 100$), le rayon est sorti de la scène : on renvoie `false`.

Au-delà de 256 itérations, on considère aussi qu'il n'y a pas d'impact. La direction $\vec{u}$ doit être normalisée, sinon t n'est plus une distance. L'origine doit être hors de l'objet.

3.2 Algorithme d'érosion

3.2.1 Projection des impacts

Les projectiles partent d'une sphère englobante de centre c et de rayon R qui contient l'objet. Pour chaque projectile, on tire une direction $\vec{v}$ uniforme sur la sphère de rayon 1 (tirage par rejet : un point uniforme dans le cube $[-1, 1]^3$ est gardé s'il est dans la boule de rayon 1, puis normalisé). Le rayon part de $c + R\,\vec{v}$ et vise le centre, avec la direction $-\vec{v}$.
Si le rayon touche l'objet en t, on crée une sphère de rayon r (le cratère) centrée au point d'impact, et on la soustrait à l'objet avec l'opérateur de soustraction : $f \leftarrow \max(f, -f_{\text{sphère}})$.
Le générateur pseudo-aléatoire est initialisé avec une graine : à graine égale, les deux méthodes ci-dessous tirent exactement les mêmes rayons, ce qui permet de les comparer.

3.2.2 Érosion incrémentale et par paquets

Incrémentale. Chaque cratère est soustrait dès qu'il est créé. Le rayon suivant est donc lancé sur l'objet déjà creusé, et il peut tomber au fond d'un cratère précédent pour creuser plus profond. L'arbre obtenu est une chaîne de n soustractions :

$$\mathrm{Sub}(\ldots \mathrm{Sub}(\mathrm{Sub}(\text{objet}, s_0), s_1) \ldots, s_{n-1}).$$

Par paquets. Les rayons sont regroupés par paquets de k. Les k rayons d'un paquet sont tous lancés sur le même objet, celui d'avant le paquet. Leurs cratères sont ensuite réunis dans un arbre d'unions équilibré (fonction `MakeUnionTree`, profondeur $\lceil \log_2 k \rceil$), qui est soustrait en une seule fois :

$$\mathrm{Sub}(\ldots \mathrm{Sub}(\text{objet}, \mathrm{Union}(\text{paquet}_0)) \ldots, \mathrm{Union}(\text{paquet}_{\lceil n/k \rceil - 1})).$$

Les rayons d'un même paquet ne voient donc pas les cratères les uns des autres. Pour $k = 1$, on retrouve exactement l'arbre incrémental.
Dans les deux cas, si les n rayons touchent l'objet, l'arbre contient n sphères et $2n + 1$ nœuds. Seule sa forme change : l'arbre incrémental a une profondeur $n + 1$, l'arbre par paquets une profondeur $\lceil n/k \rceil + \lceil \log_2 k \rceil + 1$.

3.3 Résultats

Protocole. L'objet est une boîte de demi-côtés 0,5, les cratères ont un rayon de 0,05, et la même graine est utilisée pour toutes les méthodes, qui tirent donc les mêmes rayons. On fait varier le nombre d'impacts n de 100 à 4000 et la taille des paquets k de 5 à 500. Pour chaque configuration, on mesure le temps de **construction** de l'objet érodé et le temps d'un appel à `Value` sur l'objet final. La précision est comparée visuellement.

3.3.1 Analyse : incrémental contre paquets

Incrémentale : plus précise, mais plus longue. Chaque rayon est lancé sur l'objet déjà creusé par tous les impacts précédents. Un projectile qui tombe dans un cratère creuse donc plus profond : c'est le comportement attendu d'une érosion, et c'est la référence de précision. En contrepartie, l'arbre est une chaîne de n soustractions, très profonde :

- la construction coûte cher, car le rayon i doit parcourir un arbre qui contient déjà les i cratères précédents. Le temps est multiplié par 5 à 7 quand n double (tableau 8) : il est quadratique, et même un peu plus, car la surface creusée demande plus de pas de sphere tracing ;
- l'objet final est aussi plus lent à évaluer, environ deux fois plus que par paquets pour le même nombre de cratères (tableau 9).

Table 8 – Temps de construction (ms) selon le nombre d'impacts n et la taille des paquets k.

n	Incrémentale	$k = 5$	$k = 10$	$k = 25$	$k = 50$	$k = 100$	$k = 500$
100	0,27	0,14	0,13	0,11	0,09		
500	8,34	4,12	3,59	3,25	2,99	2,64	
1000	42,7	20,9	18,5	16,4	15,6	14,5	6,94
2000	223	111	99,3	89,6	85,0	81,1	54,5
4000	1618	824	754	673	608	569	356

Table 9 – Temps d'un appel à `Value` sur l'objet final (μs).

n	Incrémentale	$k = 5$	$k = 10$	$k = 25$	$k = 50$	$k = 100$	$k = 500$
100	0,86	0,41	0,37	0,37	0,38		
500	4,44	2,21	1,92	1,79	1,69	1,69	
1000	9,10	4,53	4,08	3,74	3,55	3,52	3,51
2000	18,0	8,86	8,17	7,44	7,17	6,98	7,06
4000	34,6	19,2	17,3	14,9	14,3	14,0	14,0

Table 10 – Gain des paquets par rapport à l'érosion incrémentale pour $n = 4000$.

k	5	10	25	50	100	500
Construction	×2,0	×2,1	×2,4	×2,7	×2,8	×4,5
`Value`	×1,8	×2,0	×2,3	×2,4	×2,5	×2,5

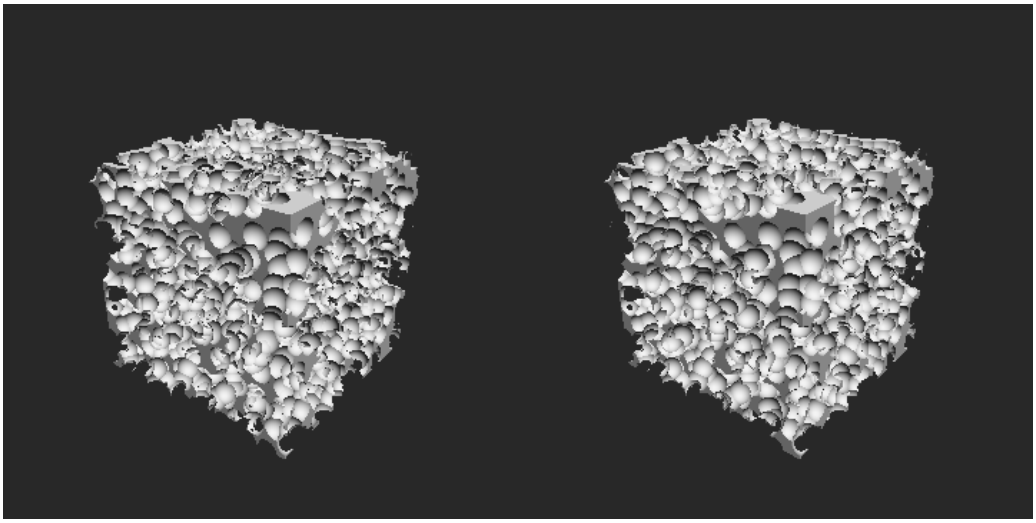


Figure 13 – Maillages des deux objets érodés obtenus par marching cubes (résolution 100), $n = 2000$ impacts, rayon des cratères 0,05, graine 42 : érosion incrémentale à gauche, paquets de $k = 500$ à droite.

Par paquets : moins précise, mais plus rapide. Les k rayons d'un paquet sont lancés sur le même objet, puis leurs cratères sont soustraits ensemble.

— *Plus rapide.* Les rayons d'un paquet parcourent un arbre qui ne contient pas encore les cratères de ce paquet. De plus, l'arbre final, une chaîne de n/k soustractions d'unions équilibrées, est beaucoup moins profond et donc plus rapide à évaluer. Pour $n = 4000$, la construction est 2 à 4,5 fois plus rapide, et `Value` 1,8 à 2,5 fois plus rapide (tableau 10).

— *Moins précise.* Les rayons d'un même paquet ne voient pas les cratères les uns des autres. Deux projectiles du même paquet qui tombent au même endroit creusent à la même profondeur au lieu de s'empiler. Dans les deux cas, la surface est criblée de cratères (figure 13), mais en incrémental les cratères s'empilent et l'érosion va plus profond dans l'objet. Plus k est grand, moins l'objet est érodé en profondeur.

Choix de la taille des paquets. Augmenter k rend la construction plus rapide, mais l'objet moins fidèle. Le gain sur `Value` plafonne dès $k \approx 25$ (×2,3 à ×2,5), alors que la perte de précision continue d'augmenter avec k. Des paquets de quelques dizaines de rayons donnent l'essentiel du gain (×2,4 à ×2,7 sur la construction, environ ×2,3 sur l'évaluation), en limitant la perte de

précision. Les deux méthodes restent quadratiques en n : les paquets divisent le temps, ils ne changent pas la façon dont il augmente avec le nombre d'impacts.

4 Accélération des requêtes

4.1 Structure et stratégie implémentées

Le volume englobant a été implémenté dans la classe `BoundingBoxSDF`. Il consiste en une approximation du champ que nous considérons suffisante étant donné que nous n'utilisons que des unions dans le cadre du test.
La méthode est simple : on renvoie la distance à la boite plus un ε en dehors de la boite, sinon on renvoie la distance à notre scène.

4.2 Résultats

Nous avons effectué des tests sur une scène de 500 sphères réparties aléatoirement dans l'espace, toutes contenues dans une boite de demi-taille 0,9.
Pour l'évaluation de la durée de la fonction `Value`, nous avons utilisé des points générés aléatoirement dans une boite de demi-taille 1.

Table 11 – Comparaison des requêtes avec et sans accélération.

Requête	Variante	`Value` (μs/appel)	FPS	Temps/frame (ms)
`Value` ou intersection	Sans volume englobant	16,66	29,9	33,6
`Value` ou intersection	Avec volume englobant	12,2	101,3	9,88

Comme on peut le voir, les résultats montrent une nette amélioration avec l'utilisation de la `BoundingBox`. Etant donné que le volume occupé par notre boite est d'environ 73% du volume dans lequel sont générés nos points, une diminution du temps d'évaluation de 16 à 12 revient à une diminution d'environ 27%, ce qui est cohérent avec la différence de volume. Ce qui revient à dire que 27% des points ont subi une amélioration proche de 100%, autrement dit, l'évaluation de la fonction ne coûte presque rien pour ces points.

5 Rendu par lancer de rayon

5.1 Sphere Tracing

La pipeline de rendu par sphère tracing est défini dans le fichier ***`sphere_tracing.cpp`***. Le fonctionnement est simple on construit un triangle occupant toute la caméra auquel on applique un shader utilisant du sphère tracinge. Le triangle agit comme un écran. L'algorithme de sphère tracing est défini dans le fichier ***`assets/SphereTracing.frag`***, les primitive et operateurs de SDF sont définis en GLSL dans le fichier ***`SDF_utils`***.
L'algorithme de sphere tracing est le même que défini précédemment. On lance un rayon puis on fait des sauts de distance/lambda (la borne lipschitzienne) jusqu'a être trop loin ou trop proche de la surface. Ici on considère que lambda est toujours égale à 1 car on n'utilise pas de domaine warping.
Pour dessiner la scene on utilise une distance maximale à partir de laquelle les rayons s'arretent. On utilise également un epsilon pour savoir si on est suffisemment proche d'une surface pour l'afficher sur ce pixel.

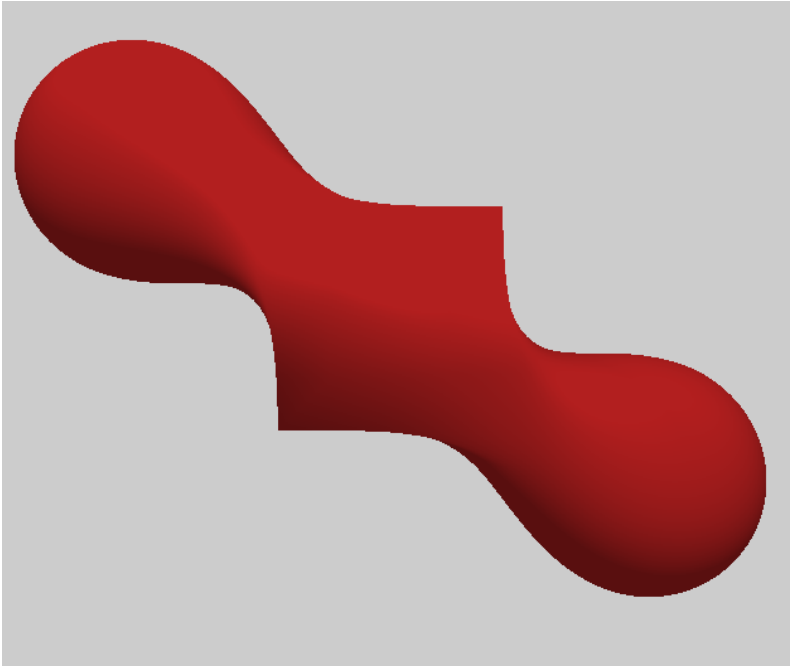


Figure 14 – Exemple de rendu sphere tracing de deux sphères et d'un cube combinés par des blends lisses.

Le rendu présente un problème au niveau des bordures des objets : l'aliasing. Cet effet est causé par le fait que l'on envoie qu'un seul rayon par pixel ce qui cause ces bordures nets et "pixelisées".On pourrait être résoudre ce problème avec une méthode d'antialiasing simple comme ajouter une proportion de la moyenne des pixels voisin.

5.2 Applatissement de l'abre

Pour utilisé un arbre de construiction défini en C++ avec cette pipeline, on utilise une méthode pour convertir l'arbre directement en code GLSL avant de le compiler en même temps que le shader. Chaque classe qui hérite de NodeSDF surcharge la méthode "ToGLSL". Cette méthode renvoie une chaine de caractères qui correspond à l'appel de value GLSL équivalent pour cette prmitive. Cette fonction assemble de manière récursive la méthode Value finale.
Cette méthode a l'avan,tage d'être simple mais elle empeche la modification de l'arbre en temps réel car il faudrait recompilé le shader généré à chaque frame.

Listing 1 – Code GLSL généré pour l'arbre de blend présenté à la figure 14.

```
blend_op(
    blend_op(
        sphere_SDF(p, vec3(-1.600000, 1.000000, 0.000000), 0.750000),
        box_SDF(p, vec3(0.000000, 0.000000, 0.000000),
                vec3(0.650000, 0.650000, 0.650000),
                vec3(0.000000, 1.000000, 0.000000)),
        0.300000),
    sphere_SDF(p, vec3(1.600000, -1.000000, 0.000000), 0.750000),
    0.300000)
```

A Code source

Le code source du projet est disponible sur GitHub : `https://github.com/egoia/SDFs`.

B Aide et ressources

Le projet a été réalisé par Elyas Saidoune et Leon Lepers. Certaines fonctions telles que la SDF du Tore ou encore la fonction de blend ont été récupérés sur le site de Inigo Qilez (`https://iquilezles.org/articles/distfunctions/`). L'IA générative a été utilisée de manière raisonnable pour générer des tests et des scenes intéressantes.